

AI Phishing Email Analyzer

Rule-Based Detection, Local-AI Guardrails and Evidence-Led Validation

Portfolio Project Report

Ayinde Perouza

August 2026

Repository: github.com/Payinde/AI-Phishing-Email-Analyzer

A defensive training application that keeps classification under transparent Python rules while using a local language model only to explain verified evidence.

1. Executive Summary

The AI Phishing Email Analyzer is a locally hosted Streamlit application developed to examine sanitized email content for phishing indicators. It combines a deterministic Python detection engine with an optional Ollama-based explanation layer. The security score, classification, indicator list, sender domain and URL count remain controlled by code; the language model cannot alter them.

The project progressed through iterative testing. Three realistic phishing simulations were initially under-classified. Those false negatives became evidence for Detection Engine v2, which added stronger social-engineering phrase coverage, multiline header parsing, sender-domain extraction and brand-domain validation. The corrected engine classified all three simulations as High risk and passed the full regression suite.

Validated outcome	Result
Automated tests	13 passed
Fresh-clone test	Passed from public GitHub repository
Legitimate synthetic sample	Low - 0/100
Outlook reactivation simulation	High - 70/100
Repository	Public, documented and MIT licensed

Security position: This is a defensive training and portfolio tool. It does not prove that an email is safe or malicious, and automated findings require human analyst validation.

2. Objectives and Scope

- Build an explainable phishing-analysis workflow rather than an opaque AI-only classifier.
- Identify suspicious language, insecure or deceptive URLs, risky file types and brand impersonation.
- Allow safe testing by paste, built-in samples or sanitized text/email uploads.
- Provide optional local-AI context without sending email content to an external cloud model.
- Produce privacy-conscious JSON evidence and a repeatable automated test suite.
- Publish a reproducible GitHub project that another user can clone, install and validate.

3. Solution Architecture

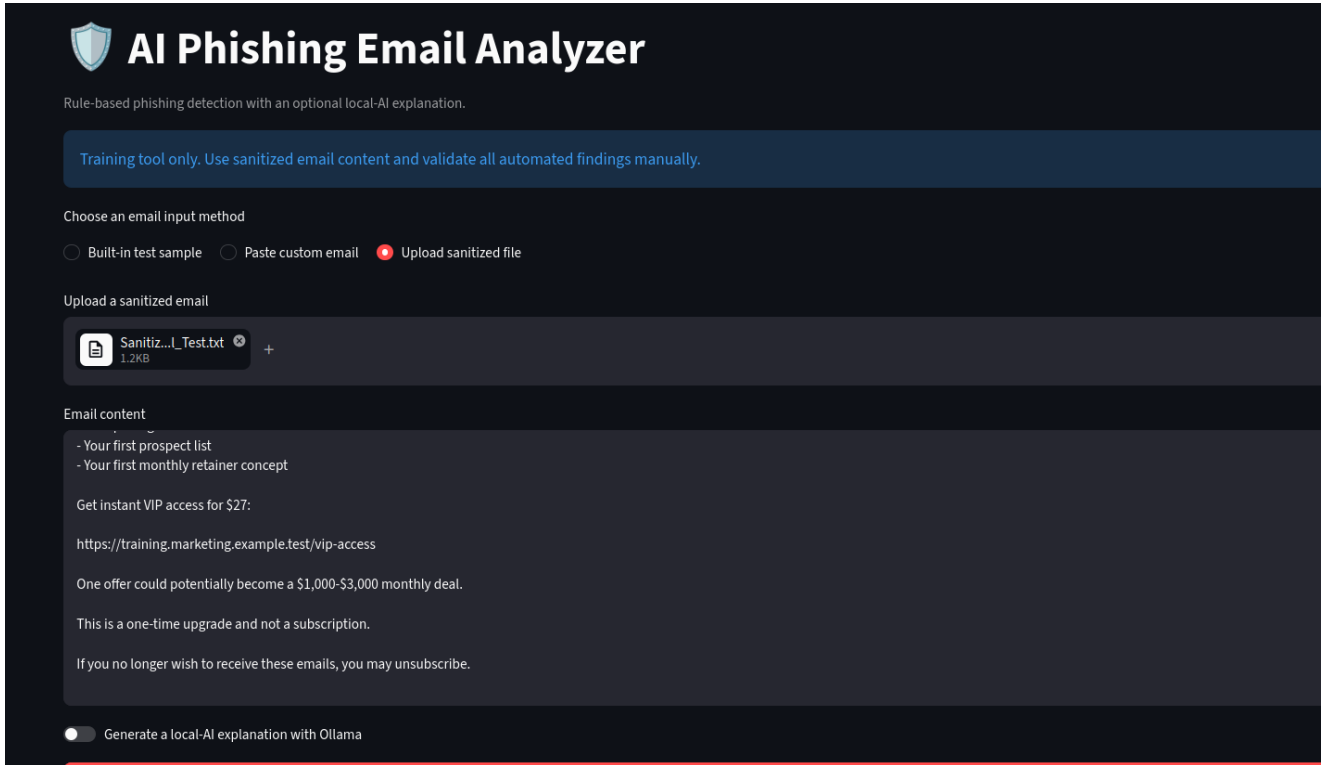
Sanitized input Built-in, paste, .txt or .eml	Detection engine Rules, headers, URLs, scoring
Optional local AI Potential-impact wording only	Streamlit output Evidence, score and JSON report

Trust boundary: Python produces verified evidence and classification. Ollama receives structured findings and may explain possible impact, but its response must pass quality checks. When the response fails or times out, a deterministic fallback is displayed.

Component	Responsibility
app.py	Streamlit interface, input handling, results presentation and JSON download
analyzer.py	Header parsing, sender-domain extraction, URL analysis, indicator scoring and risk classification
ai_explainer.py	Local Ollama execution, output cleanup, quality validation and safe fallback
tests/	Regression checks for detection logic and AI guardrails
samples/	Safe synthetic Low- and High-risk email content

4. Application Interface Walkthrough

The Streamlit interface was designed so a reviewer can understand where the email came from, what the engine detected and whether the optional local model was used. The interface supports a built-in sample, pasted content and sanitized file upload.



The screenshot shows the 'AI Phishing Email Analyzer' interface. At the top, there's a title 'AI Phishing Email Analyzer' with a shield icon. Below it, a subtitle reads 'Rule-based phishing detection with an optional local-AI explanation.' A blue banner contains the text 'Training tool only. Use sanitized email content and validate all automated findings manually.' Under 'Choose an email input method', three radio buttons are shown: 'Built-in test sample' (unselected), 'Paste custom email' (unselected), and 'Upload sanitized file' (selected). The 'Upload a sanitized email' section shows a file named 'Sanitized..._Test.txt' (1.2KB) with a plus sign. The 'Email content' section displays a sample phishing email body. At the bottom, a toggle switch for 'Generate a local-AI explanation with Ollama' is shown, currently turned off.

AI Phishing Email Analyzer

Rule-based phishing detection with an optional local-AI explanation.

Training tool only. Use sanitized email content and validate all automated findings manually.

Choose an email input method

☐ Built-in test sample ☐ Paste custom email ☒ Upload sanitized file

Upload a sanitized email

Sanitized..._Test.txt 1.2KB +

Email content

- Your first prospect list
- Your first monthly retainer concept

Get instant VIP access for \$27:

<https://training.marketing.example.test/vip-access>

One offer could potentially become a \$1,000-\$3,000 monthly deal.

This is a one-time upgrade and not a subscription.

If you no longer wish to receive these emails, you may unsubscribe.

☐ Generate a local-AI explanation with Ollama

Figure 1 - Main application interface showing the training disclaimer, input methods, sanitized upload, email content and optional Local-AI toggle.

Low-risk presentation: A legitimate sample produces a green status message, a 0/100 score, no configured indicators and no extracted URLs.

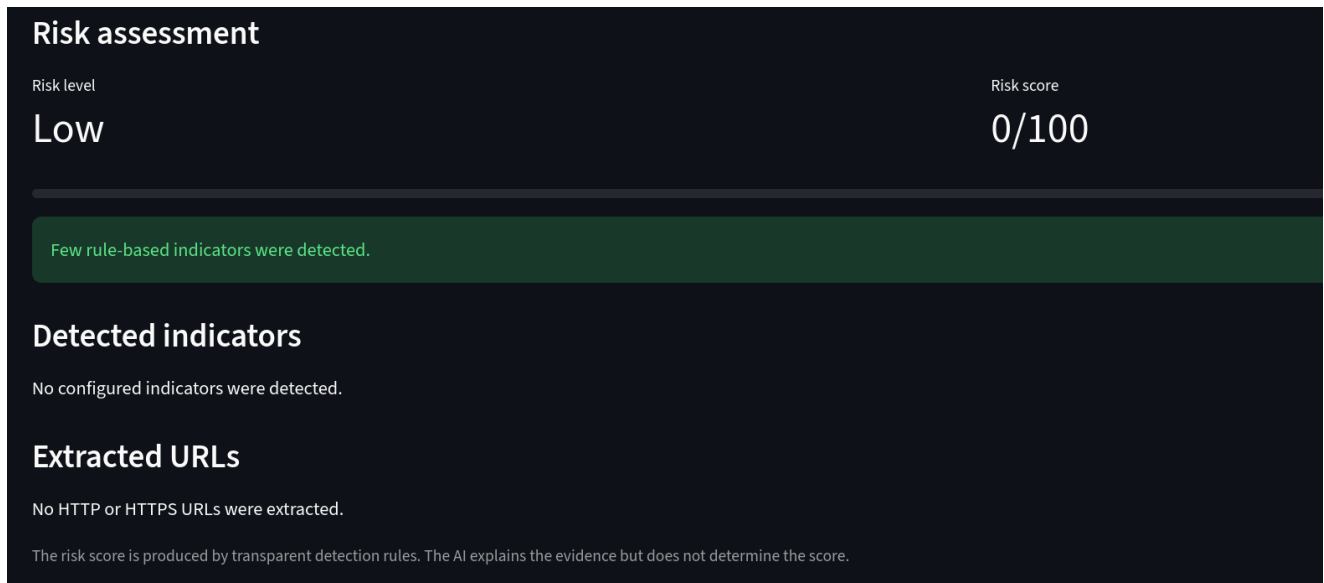


Figure 2 - Built-in legitimate sample classified as Low risk, 0/100.

Medium-risk presentation: Suspicious account language produces a review warning and an evidence table while avoiding an automatic High classification.

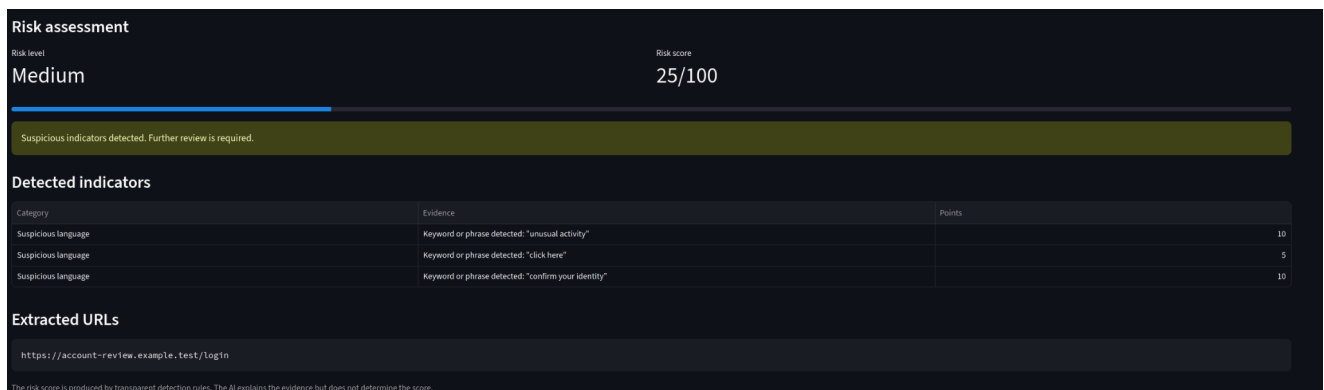


Figure 3 - Suspicious account-review sample classified as Medium risk, 25/100.

High-risk presentation: Multiple independent indicators produce a clear escalation message. The interface displays the score contribution of every rule, extracted URLs and the optional explanation.

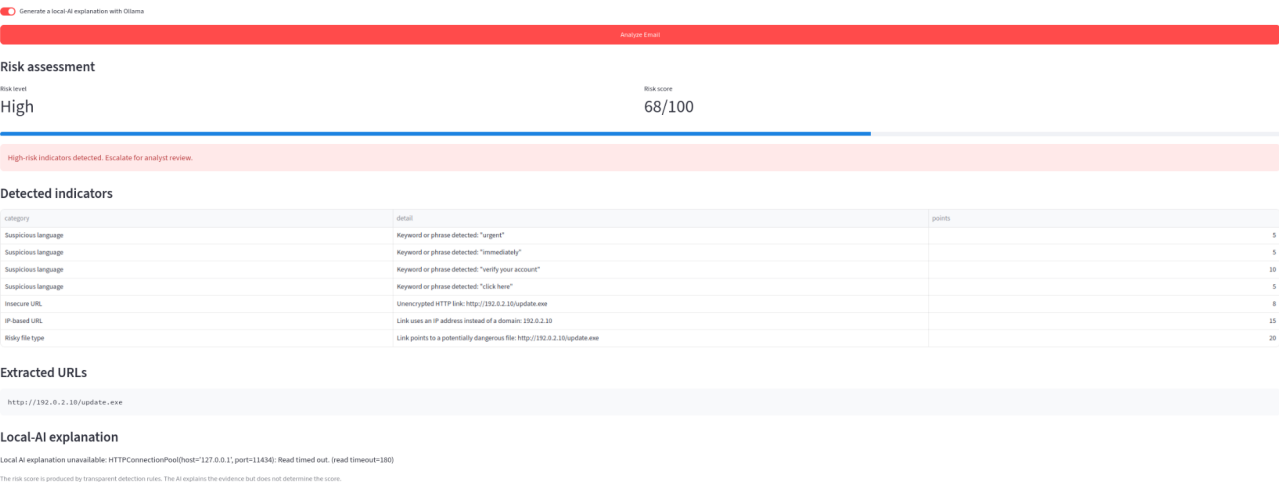


Figure 4 - Generic phishing simulation classified as High risk, 68/100, with seven transparent indicators.

The final application also displays the extracted sender domain and provides a button to download the sanitized JSON analysis report. Raw email content is intentionally excluded from that report.

5. Detection Logic

- Suspicious language: urgency, account verification, password expiration, refund and reactivation phrases.
- Sender analysis: parses sender headers, including multiline formatting, and extracts the sender domain.
- Brand impersonation: compares Microsoft or HMRC language with the claimed sender domain.
- URL inspection: identifies HTTP, IP-based, shortened and punycode URLs plus risky file extensions.
- Transparent scoring: indicator points are summed, capped at 100 and mapped to Low, Medium or High.

6. Build Process and Key Corrections

Issue observed	Cause / learning	Correction implemented
Local-AI explanation timed out	The local model could take 35-48 seconds and HTTP calls sometimes exceeded the configured limit.	Used safe subprocess execution, a smaller qwen2.5:1.5b model, bounded retries and a deterministic fallback.
Terminal control characters appeared in UI	Model output contained ANSI/control sequences.	Added output cleaning before displaying AI text.
AI produced duplicated or unsupported wording	The model had too much responsibility for structured findings.	Python now generates evidence, exact URL counts and validation statements; AI generates only potential-impact wording.
Malformed phrase such as "risk d due"	The model emitted an invalid single-letter fragment.	Added single-letter and repetition quality guards; rejected output falls back safely.
Three phishing simulations were under-classified	Rules did not yet recognize the full social-engineering and impersonation patterns.	Detection Engine v2 expanded phrases, corrected header parsing and added brand-domain validation.
test/test_analyzer.py could not be saved	The target directory was named tests, not test.	Created and used tests/test_analyzer.py, then validated with unittest discovery.
python: command not found in a new terminal	The virtual environment was not active in the new shell.	Activated it with source .venv/bin/activate and repeated validation.
Git and GitHub CLI unavailable	The Ubuntu VM did not have the tools installed.	Installed git and gh, authenticated through GitHub device login, committed and pushed securely.

The correction workflow followed a useful engineering pattern: reproduce the problem, identify the cause, rewrite the affected component where necessary, compile or execute a targeted check, then rerun the full regression suite.

7. Detection Engine v2: Evidence-Led Improvement

The first expanded regression run exposed three false negatives. The legitimate, generic phishing, score-cap and suspicious-email tests passed, but all three brand-impersonation simulations returned Medium instead of the expected High classification.

```
(.venv) osboxes@wcds-app-01:~/ai-phishing-email-analyzer$ python -m unittest discover -s tests -v
test_hmrc_refund_impersonation_is_high_risk (test_analyzer.TestPhishingAnalyzer.test_hmrc_refund_impersonation_is_high_risk) ... FAIL
test_legitimate_email_is_low_risk (test_analyzer.TestPhishingAnalyzer.test_legitimate_email_is_low_risk) ... ok
test_outlook_reactivation_impersonation_is_high_risk (test_analyzer.TestPhishingAnalyzer.test_outlook_reactivation_impersonation_is_high_risk) ... FAIL
test_password_expiration_impersonation_is_high_risk (test_analyzer.TestPhishingAnalyzer.test_password_expiration_impersonation_is_high_risk) ... FAIL
test_phishing_simulation_is_high_risk (test_analyzer.TestPhishingAnalyzer.test_phishing_simulation_is_high_risk) ... ok
test_score_cannot_exceed_100 (test_analyzer.TestPhishingAnalyzer.test_score_cannot_exceed_100) ... ok
test_suspicious_email_is_medium_risk (test_analyzer.TestPhishingAnalyzer.test_suspicious_email_is_medium_risk) ... ok

=====
FAIL: test_hmrc_refund_impersonation_is_high_risk (test_analyzer.TestPhishingAnalyzer.test_hmrc_refund_impersonation_is_high_risk)
-----
Traceback (most recent call last):
  File "/home/osboxes/ai-phishing-email-analyzer/tests/test_analyzer.py", line 72, in test_hmrc_refund_impersonation_is_high_risk
    self.assertEqual(result["risk_level"], "High")
AssertionError: 'Medium' != 'High'
- Medium
+ High

=====
FAIL: test_outlook_reactivation_impersonation_is_high_risk (test_analyzer.TestPhishingAnalyzer.test_outlook_reactivation_impersonation_is_high_risk)
-----
Traceback (most recent call last):
  File "/home/osboxes/ai-phishing-email-analyzer/tests/test_analyzer.py", line 122, in test_outlook_reactivation_impersonation_is_high_risk
    self.assertEqual(result["risk_level"], "High")
AssertionError: 'Medium' != 'High'
- Medium
+ High

=====
FAIL: test_password_expiration_impersonation_is_high_risk (test_analyzer.TestPhishingAnalyzer.test_password_expiration_impersonation_is_high_risk)
-----
Traceback (most recent call last):
  File "/home/osboxes/ai-phishing-email-analyzer/tests/test_analyzer.py", line 98, in test_password_expiration_impersonation_is_high_risk
    self.assertEqual(result["risk_level"], "High")
AssertionError: 'Medium' != 'High'
- Medium
+ High

-----
Ran 7 tests in 0.005s

FAILED (failures=3)
(.venv) osboxes@wcds-app-01:~/ai-phishing-email-analyzer$
```

Figure 5 - Initial regression run showing three impersonation tests failing.

Simulation	Initial	Final	Primary improvement
HMRC tax refund	Low - 8	High - 68	Refund phrases + HMRC/domain mismatch
Microsoft password expiry	Low - 13	High - 88	Expiry language + Microsoft/domain mismatch
Outlook account reactivation	Low - 0	High - 70	Reactivation language + Microsoft/domain mismatch

8. Corrected High-Risk Results

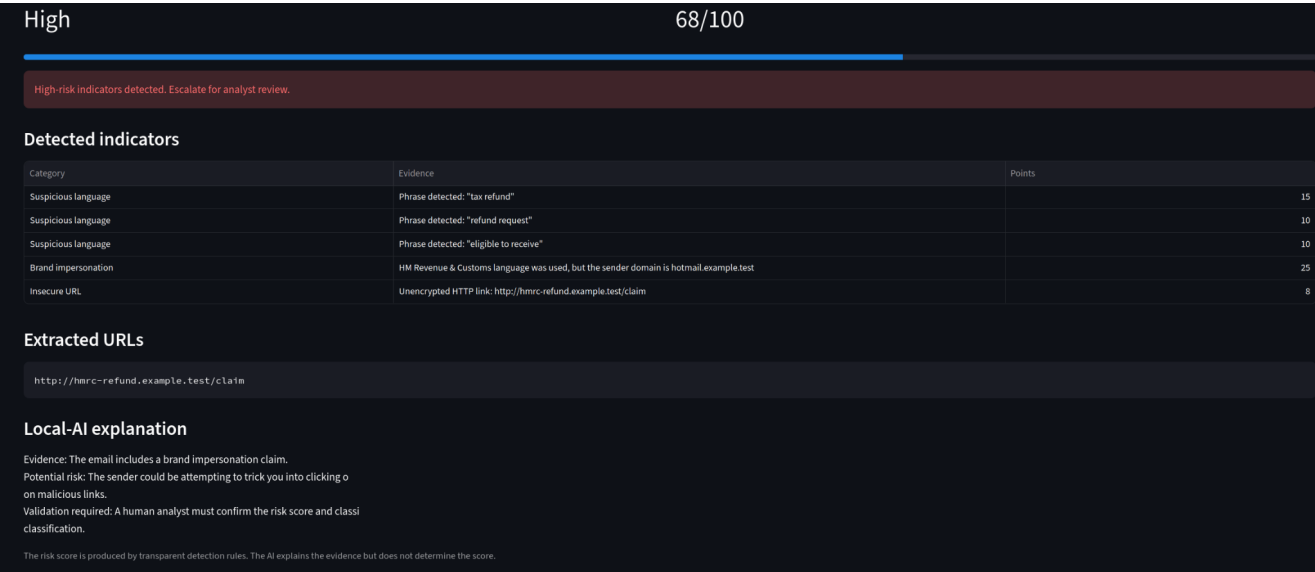


Figure 6 - HMRC refund simulation corrected to High risk, 68/100.

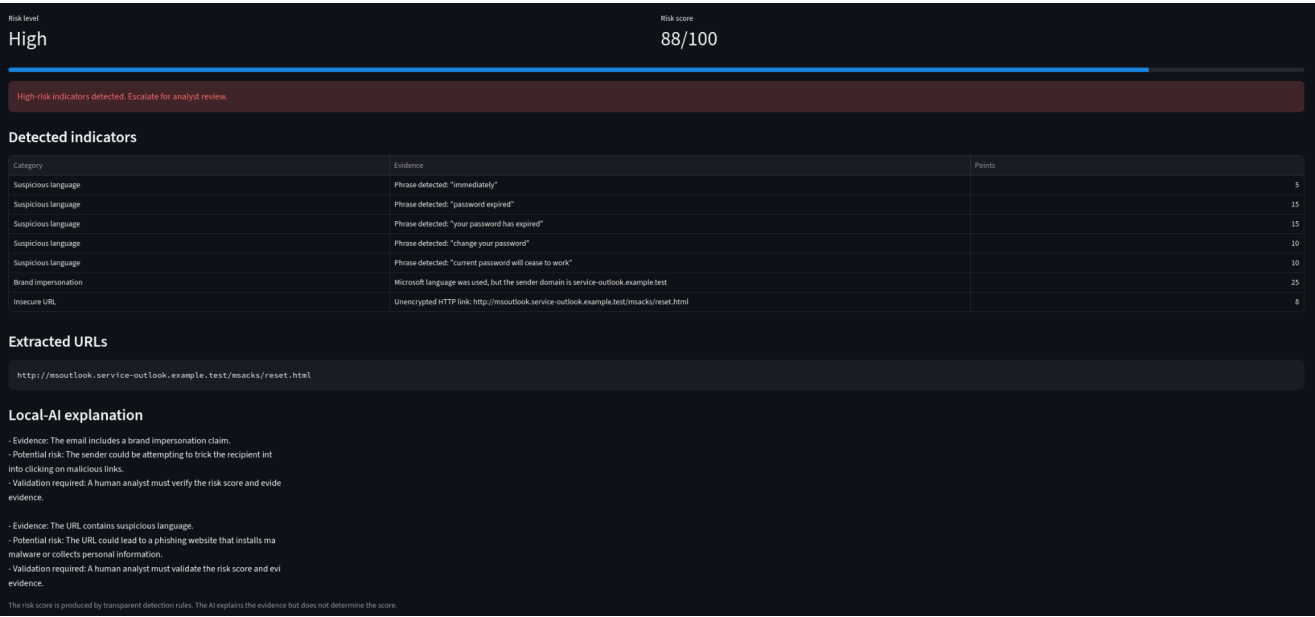


Figure 7 - Password-expiration simulation corrected to High risk, 88/100.

9. Final Outlook Reactivation Validation

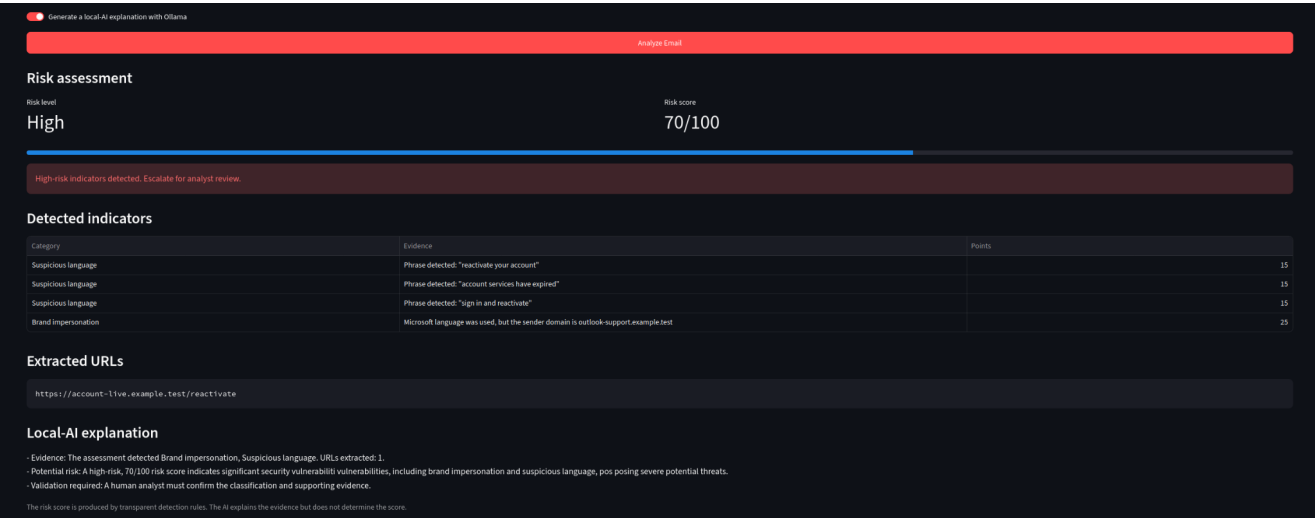


Figure 8 - Final Outlook reactivation result: High, 70/100, four indicators and one extracted URL.

The deterministic indicators totalled 70 points: three suspicious-language findings at 15 points each and one Microsoft brand-impersonation finding at 25 points. The local-AI explanation reported one URL and did not change the classification.

10. Complete Email Test Matrix

Eight principal scenarios were exercised through the interface or directly through the analyzer. The set deliberately included legitimate and marketing content so the project could demonstrate discrimination rather than simply assigning a high score to every message.

Scenario	Purpose	Final result	Evidence / learning
Built-in HR notice	Baseline legitimate message	Low - 0/100	No configured indicators; no URL
Account-review notice	Moderately suspicious language	Medium - 25/100	Three phrases; HTTPS URL alone added no URL-risk points
Generic IP/.exe lure	Multiple independent phishing signals	High - 68/100	Urgency, verification, HTTP, IP host and executable path
Sanitized Loom notice	Known legitimate message	Low - 0/100	Demonstrated that a normal HTTPS link is not automatically suspicious
Marketing/VIP offer	Spam/marketing distinction	Low - 0/100	Unwanted marketing is not automatically phishing
HMRC tax refund	Government-brand impersonation	High - 68/100	Initially Low 8; corrected with phrases and sender-domain mismatch
Microsoft password expiry	Credential-theft impersonation	High - 88/100	Initially Low 13; corrected with expiry and brand rules
Outlook reactivation	Account-reactivation impersonation	High - 70/100	Initially Low 0; corrected with phrases and sender-domain mismatch

Interpretation: A Low result means no configured rule reached the Medium threshold. It does not prove that the message is legitimate. A human analyst must still consider context, authentication headers, sender history and business expectations.

11. Sanitized Email Samples Used

The following content is sanitized portfolio evidence. Reserved .test domains prevent the examples from directing readers to active sites.

Sample 1 - Built-in legitimate HR notice | Final: Low 0/100

From: HR Department <hr@example.test>
To: employee@example.test
Subject: Updated holiday schedule

Hello team,

The updated holiday schedule is available on the internal employee portal.
Please contact Human Resources if you have any questions.

Regards,
HR Department

Sample 2 - Suspicious account review | Final: Medium 25/100

From: Account Review <notice@account-review.example.test>
To: user@example.test
Subject: Unusual activity detected

We noticed unusual activity on your account.
Click here to confirm your identity:
<https://account-review.example.test/login>

Sample 3 - Generic IP and executable lure | Final: High 68/100

From: Security Team <alert@example.test>
To: user@example.test
Subject: URGENT account verification

URGENT: Verify your account immediately.
Click here: <http://192.0.2.10/update.exe>

Sample 4 - Sanitized Loom account notice | Final: Low 0/100

From: Loom <notifications@loom.example.test>
To: user@example.test
Subject: Action needed to keep your Loom account active

Your Loom account is scheduled for routine account maintenance.
Review your account information at:
<https://login.loom.example.test/account>

Sample 5 - Sanitized marketing/VIP offer | Final: Low 0/100

Subject: Build your first recurring offer

- Your first prospect list
- Your first monthly retainer concept

Get instant VIP access for \$27:
<https://training.marketing.example.test/vip-access>

One offer could potentially become a \$1,000-\$3,000 monthly deal.
This is a one-time upgrade and not a subscription.
If you no longer wish to receive these emails, you may unsubscribe.

Sample 6 - HMRC tax-refund impersonation | Final: High 68/100

From: HM Revenue & Customs <refund@hmrc-support.example.test>
To: user@example.test
Subject: Support Center: Tax refund

New Tax Calculation
You are eligible to receive a tax refund of 209.27 GBP.
Please submit the tax refund request:
<http://hmrc-refund.example.test/claim>

Sample 7 - Microsoft password-expiration impersonation | Final: High 88/100

From: Microsoft Outlook <msoutlook94@service-outlook.example.test>
To: user@example.test
Subject: Your Password Has Expired

Your password has expired.
For your account security, your current password will cease to work shortly.
Change your password immediately:
<http://msoutlook.service-outlook.example.test/msacks/reset.html>

Sample 8 - Outlook account-reactivation impersonation | Final: High 70/100

From: Microsoft Account Team <account-team@outlook-support.example.test>
To: user@example.test
Subject: Reactivate Your Outlook Account

Dear User,

Your Hotmail account services have expired.
You must reactivate your account for it to remain active.
Sign in and reactivate your account:
<https://account-live.example.test/reactivate>

Thanks,
The Microsoft Account Team

These examples are included for transparent reproduction. They should be saved as plain text and analyzed without visiting any URL.

12. Local-AI Guardrails and Graceful Degradation

Ollama is optional. The application remains useful when the model is disabled, unavailable or slow. This design prevents an AI outage from removing the core rule-based assessment.

Controlled by Python	Allowed AI role
Risk score and classification	Explain possible impact cautiously
Detected categories and indicator details	Use only supplied categories
Exact URL count	Reference the deterministic count
Human-validation statement	No authority to remove or weaken it

Rejected AI conditions: unsupported claims, incorrect multiple-link claims, repeated words, malformed single-letter fragments, unexpected headings, excessive length and overconfident conclusions.

Validated fallback: “The combined indicators may represent phishing, credential theft, malware delivery, or fraud.”

13. Automated and Fresh-Clone Validation

The final suite contained seven analyzer tests and six AI guardrail tests. It passed locally and again from a clean clone of the public GitHub repository.

Validation area	Evidence
Risk classification	Legitimate Low, suspicious Medium, phishing and impersonation High
Score safety	Total cannot exceed 100
Parsing	Sender domain, multiline headers and URLs
AI quality	Repetition, unsupported claims and malformed fragments rejected
Reproducibility	13 tests passed from a new temporary clone

```
osboxes@wcds-app-01:~/ai-phishing-email-analyzer$ source .venv/bin/activate
(.venv) osboxes@wcds-app-01:~/ai-phishing-email-analyzer$ python - <<'PY'
from pathlib import Path
from analyzer import analyze_email

for path in sorted(Path("samples").glob("*.txt")):
    result = analyze_email(path.read_text(encoding="utf-8"))
    print(
        f"{path.name}: "
        f"{result['risk_level']} | "
        f"{result['score']}/100 | "
        f"sender={result['sender_domain']} | "
        f"indicators={len(result['indicators'])} | "
        f"urls={len(result['urls'])}"
    )
PY
legitimate_email.txt: Low | 0/100 | sender=example.test | indicators=0 | urls=0
outlook_reactivation_phishing.txt: High | 70/100 | sender=outlook-support.example.test | indicators=4 | urls=1
```

Figure 9 - Published samples validated directly: legitimate Low 0/100 and Outlook High 70/100.

14. Reporting and Privacy Controls

- The JSON report records UTC time, risk level, score, sender domain, indicators, URLs, AI status, explanation and disclaimer.
- Raw email content is excluded from the downloaded JSON report to reduce unnecessary retention.
- Synthetic .test domains prevent demonstration links from pointing to active infrastructure.
- The application extracts URLs as text; it does not visit or execute them.
- Uploads and results are intended for sanitized training material only.

15. Git and GitHub Publication Workflow

The project was converted from a local folder into a reproducible public repository using the normal Git lifecycle.

Stage	Command / action	Purpose
Initialize	git init -b main	Begin local version tracking on the main branch
Inspect	git status, dry-run and secret scan	Confirm only approved files would be committed
Commit	git add . and git commit	Create the first versioned checkpoint
Connect	git remote add origin	Associate the VM project with GitHub
Authenticate	gh auth login	Use secure browser/device authentication
Publish	git push -u origin main	Upload and track the public main branch
License	MIT License	Permit use, study and modification with attribution
Reproduce	Fresh clone + venv + tests	Confirm another visitor can run the project

```
osboxes@wcds-app-01:~/ai-phishing-email-analyzer$ ^C
osboxes@wcds-app-01:~/ai-phishing-email-analyzer$ git add .
osboxes@wcds-app-01:~/ai-phishing-email-analyzer$ git status --short
A .gitignore
A README.md
A ai_explainer.py
A analyzer.py
A app.py
A evidence/README.md
A requirements.txt
A samples/legitimate_email.txt
A samples/outlook_reactivation_phishing.txt
A tests/test_ai_explainer.py
A tests/test_analyzer.py
osboxes@wcds-app-01:~/ai-phishing-email-analyzer$ git diff --cached --stat
.gitignore      | 5 +++
README.md       | 466 +++++
ai_explainer.py | 311 +++++
analyzer.py     | 342 +++++
app.py          | 309 +++++
evidence/README.md | 33 +++++
requirements.txt | 1 +
samples/legitimate_email.txt | 14 +++++
samples/outlook_reactivation_phishing.txt | 19 +++++
tests/test_ai_explainer.py | 127 +++++
tests/test_analyzer.py | 152 +++++
11 files changed, 1779 insertions(+)
osboxes@wcds-app-01:~/ai-phishing-email-analyzer$
```

Figure 10 - Approved project files staged; virtual environment and secrets excluded.

16. Reproduction Guide

A visitor can reproduce the validated rule-based application without Ollama:

```
git clone https://github.com/Payinde/AI-Phishing-Email-Analyzer.git
cd AI-Phishing-Email-Analyzer
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
streamlit run app.py
```

For optional local explanations, install Ollama, pull qwen2.5:1.5b and enable the Local-AI toggle in Streamlit.

17. Annotated Implementation Excerpts

These abridged excerpts show the main implementation patterns. The complete, current scripts are maintained in the public GitHub repository.

A. Deterministic analysis pipeline

```
def analyze_email(email_text):
    headers = extract_headers(email_text)
    sender_domain = extract_sender_domain(headers)
    urls = extract_urls(email_text)
    indicators = detect_language(email_text)
    indicators += analyze_urls(urls)
    indicators += detect_brand_impersonation(email_text, sender_domain)
    score = min(sum(item["points"] for item in indicators), 100)
    risk_level = classify_score(score)
    return {"risk_level": risk_level, "score": score,
            "sender_domain": sender_domain, "urls": urls,
            "indicators": indicators}
```

Why: Evidence collection is completed before classification. The score is transparent, reproducible and capped at 100.

B. Risk thresholds

```
def classify_score(score):
    if score >= 50:
        return "High"
    if score >= 25:
        return "Medium"
    return "Low"
```

Why: Fixed thresholds make classification explainable and easy to regression-test.

C. Safe local Ollama execution

```
completed = subprocess.run(
    ["ollama", "run", MODEL_NAME],
    input=prompt,
    text=True,
    capture_output=True,
    timeout=timeout_seconds,
    env=environment,
    check=False,
)
```

Why: The argument list avoids shell interpolation. Timeout handling prevents the optional explanation from blocking the core assessment indefinitely.

D. AI quality gate and fallback

```
candidate = request_ai_risk_sentence(result)
if not ai_sentence_is_acceptable(candidate, result):
    candidate = build_default_risk_sentence(result)

return "\n".join([
    build_evidence_sentence(result),
    f"- Potential risk: {candidate}",
    "- Validation required: A human analyst must confirm "
    "the classification and supporting evidence.",
])
```

Why: Deterministic Python retains control of evidence and validation. Rejected AI output cannot overwrite verified findings.

E. Privacy-conscious JSON report

```
report = {
    "report_title": "AI Phishing Email Analysis",
    "generated_at_utc": datetime.now(timezone.utc).isoformat(),
    "risk_level": result["risk_level"],
    "risk_score": result["score"],
    "sender_domain": result["sender_domain"],
    "indicators": result["indicators"],
    "extracted_urls": result["urls"],
    "local_ai_enabled": use_local_ai,
    "explanation": explanation,
    "disclaimer": "Automated findings require human analyst validation.",
}
# Raw email content is deliberately not included.
```

Why: The exported evidence is useful for portfolio review while reducing unnecessary retention of potentially sensitive message content.

F. Regression test pattern

```
def test_outlook_reactivation_impersonation_is_high_risk(self):
    result = analyze_email(OUTLOOK_SAMPLE)
    self.assertEqual(result["risk_level"], "High")
    self.assertEqual(result["score"], 70)
    self.assertEqual(result["sender_domain"],
        "outlook-support.example.test")
```

Why: A former false negative becomes a permanent test, preventing the same detection gap from silently returning.

Complete source: <https://github.com/Payinde/AI-Phishing-Email-Analyzer>

18. Limitations and Responsible Use

- Rule coverage is intentionally limited and must be expanded as new patterns are validated.
- A Low result does not prove legitimacy; a High result does not independently prove malicious intent.
- SPF, DKIM and DMARC are not fully validated, and reputation services are not queried.
- Attachments are not executed or dynamically analyzed.
- Local model quality and response time depend on the available hardware.
- The Streamlit development server should not be exposed directly to the public Internet.
- All automated findings require human analyst review.

19. Learning Outcomes

- Python security application development and modular design.
- Email header, sender-domain and URL parsing.
- Transparent detection scoring and false-negative remediation.
- Regression testing and evidence-led iteration.
- Safe local-AI integration with constrained responsibilities.
- Privacy-conscious reporting and sanitized test data.
- Git version control, GitHub authentication, licensing and fresh-clone validation.

20. Conclusion

The AI Phishing Email Analyzer demonstrates a practical and defensible approach to AI-assisted security tooling. Its main value is not simply that it produces a score; it shows how deterministic evidence, local AI, quality controls, automated testing and analyst validation can be combined without handing security decisions to a language model.

The project also documents an important engineering lesson: the first version did not correctly classify every realistic phishing simulation. Those failures were retained as evidence, investigated and converted into regression tests. Detection Engine v2 then corrected the gaps and passed the expanded suite.

A successful fresh-clone test confirmed that the public repository can be installed and validated by another user. The repository is documented, MIT licensed and ready to support portfolio review or further defensive development.

Project repository

<https://github.com/Payinde/AI-Phishing-Email-Analyzer>

Portfolio

<https://payinde.github.io>

LinkedIn

<https://www.linkedin.com/in/ayinde-perouza>

Prepared as portfolio evidence for defensive cybersecurity learning and practical development. All examples use sanitized or synthetic content.